

BS1 B 系列

抓資料與格式解剖： 公開資料實戰

從一串 accession 到一個能分析的矩陣——資料地圖、下載紀律、七種格式、一套尺度判讀。

約 35 分鐘 | 前置：B1、B5 | B 系列補充第 1 集

同一個 GSE ，三個人，三種檔案



示意圖

資料沒有壞、網站沒有騙人——三個人都只是「不認得檔案的長相」

三個人都卡住了——但資料沒有壞，網站也沒有騙人

本集的起點

**卡住你的不是資料，
是你不認得它的長相。**

公開資料的技能不是「會下載」，是認得每一種檔案、知道用哪把鑰匙開。這一集就是那串鑰匙。

這一集你會學到

- 1 看懂公開資料的地圖：三層入口、accession 對照表、raw 與 processed 兩層
- 2 養成下載紀律：GEO 導航、wget -c、md5 驗證、看到管制存取就繞路
- 3 解剖七種矩陣格式、判讀三種尺度，把每個決定寫進 decisions.md

01

公開資料地圖

先知道世界長什麼樣子，再談下載

三層入口

公開資料的三層入口

越上層越權威完整、越下層越方便；找資料由下往上問

第一層 儲存庫（原始出處）

GEO

>20 萬 studies (NAR 2024)

BioStudies

E-MTAB 的家（歐系）

GSA

CRA 公開／HRA 管制

SRA

raw FASTQ 專區

第二層 圖譜站／整理站（先看再下載）

CELLxGENE

人類 unique 99.6M cells

EBI SCEA

同一條管線重算 1,000 萬 cells

UCSC CB / SCP

看作者原始 UMAP 與註解

HCA / DISCO

圖譜級專案／深度整合

第三層 一行載入（課堂與練習）

SeuratData

pbmc3k、ifnb...

scRNAseq (Bioc)

fetchDataset() 回 SCE

sc.datasets

Python 內建教學集

10x Datasets

官方示範資料 (CC BY 4.0)

先查下層

儲存庫最權威、圖譜站先看再抓、一行載入最省事——找資料由下往上問

三層入口怎麼用

數字出自參考庫 2026-09 查證快照；使用前以官方頁為準

層級	代表入口	什麼時候用
儲存庫	GEO（>20 萬 studies）、BioStudies、GSA	手上有 accession 時直達；題庫絕大多數題的家
圖譜站／整理站	CELLxGENE（99.6M unique cells）、EBI SCEA、UCSC CB、Broad SCP	找「新」資料、先看作者註解與 UMAP 再決定下載
一行載入	SeuratData、scRNAseq、sc.datasets、10x Datasets	課堂示範與練習：pbmc3k、ifnb 這些老朋友

看 accession 認出處

看 accession 認出處

論文只給一串代號時，用前綴反查該去哪裡下載

GSE12345	→	GEO	processed 矩陣在 Supplementary
SRR / SRP	→	SRA	raw FASTQ——再分析用不到
E-MTAB-1234	→	BioStudies / ArrayExpress	歐系實驗室常態；FTP 直下
SCP259	→	Broad Single Cell Portal	下載可能要 Google 登入
CRA001160	→	GSA (CNCB)	公開，但原始檔多為 FASTQ
HRA...	→	GSA-Human	controlled access——看到就繞路
E-CURD-46	→	EBI SCEA	同一條管線的重算版矩陣

論文只給一串代號時，前綴就是路標——這張表值得存下來



accession 對照表

同一份研究可能同時有多個代號——選 processed 那一個

前綴	去哪裡	注意
GSE	GEO	processed 在頁底 Supplementary files
SRR / SRP	SRA	raw FASTQ ； 再分析用不到
E-MTAB	BioStudies / ArrayExpress	歐系常態； FTP 直下免登入
SCP	Broad Single Cell Portal	下載可能要求 Google 登入
CRA	GSA （ CNCB ）	公開，但原始檔多為 FASTQ
HRA	GSA-Human	controlled access—— 看到就繞路
E-CURD	EBI SCEA	同一條管線重算的可比矩陣

raw 與 processed：兩層鐵則

同一份研究，兩層資料

再分析只要 processed 層——不要為了 FASTQ 去申請管制存取

raw 層：FASTQ 原始讀序



- 住在 SRA／EGA／dbGaP／GSA
- 動輒數十到數百 GB，常有管制
- 只有重跑對齊（B2-B3）才需要

看到 dbGaP／HRA／
DUOS／EGA？

那是 raw 的管制門。
繞路：回 GEO 找處理版，
或讀 Data availability
找 Zenodo／Dryad／figshare

processed 層：counts 矩陣



- 住在 GEO Supplementary、圖譜站、期刊附檔
- MB 到 GB 級，多數免登入
- 題庫與多數再分析只需要這層

每次下載記三件事

入口 + release 版本
accession
下載日期

raw 住管制區、動輒數十 GB；processed 住 Supplementary、多數免登入

本段鐵則

再分析只要 processed 。

B2-B3 教過從 FASTQ 重跑——那用小型示範資料就夠。不要為了下載 FASTQ 去申請管制存取。

02

下載實務

GEO 導航、斷點續傳、md5、大檔策略、管制辨識

GEO 頁怎麼走

GEO 頁怎麼走、大檔怎麼抓



① 先抓最小的檔

metadata + filelist 確認內容，再決定抓大檔

② wget -c 斷點續傳

大檔斷線不用重來；磁碟先預留足夠空間

③ md5sum 驗完整

跟頁面或 filelist 的 md5 對上才算下載完成

`acc.cgi?acc=GSE...` 開頁、拉到最底—— Supplementary files 才是你的目的地

下載三件組： `wget -c` + `md5sum`

```
# 大檔斷點續傳：斷線重跑同一行就會接著抓
wget -c "https://ftp.ncbi.nlm.nih.gov/geo/.../GSExxx_RAW.tar"

md5sum GSExxx_RAW.tar # 跟頁面 /filelist 的 md5 對
tar -tf GSExxx_RAW.tar | head # 解開前先看內容清單
```

```
a3f9...e21c GSExxx_RAW.tar ← 對上才算下載完成
GSM000001_sample1.matrix.gz
GSM000001_sample1.features.tsv.gz
GSM000001_sample1.barcodes.tsv.gz
```

`-c` 是 `continue`：斷線不用從零開始；`md5` 對不上就重抓，不要硬讀

大檔策略三招

先抓最小的檔



metadata + filelist
先確認內容合用，再
決定抓大矩陣（空間
資料同款紀律）

RAW.tar 可以拆買



逐樣本包裹不必整包
扛——進單一 GSM
頁抓單一樣本的檔就
好

磁碟先算再抓



題庫的粗估：每題預
留 5-10 GB；解壓
與轉檔都要工作空間

管制存取：認得門牌就好

雷是什麼

dbGaP、EGA、DUOS、Synapse、HRA——這些名字出現時，那份資料要申請、審批、簽協議。



為什麼會踩

它們管的幾乎都是 raw 定序資料的個資風險；processed 矩陣通常另有免登入的家。

踩了會怎樣

正確反應不是去申請，是繞路：回 GEO 找處理版，或讀論文 Data availability 找 Zenodo / Dryad / figshare。

繞路實戰：一個真實案例

正門：GSA 只有 FASTQ



胰臟癌 CRA001160
—— 原始檔需登入、
且全是 FASTQ（題
庫進階 13 的場景）

讀 Data availability



作者把處理後 h5ad
放在 Zenodo
（record
3969339）——免
登入直下

繞路也要驗貨



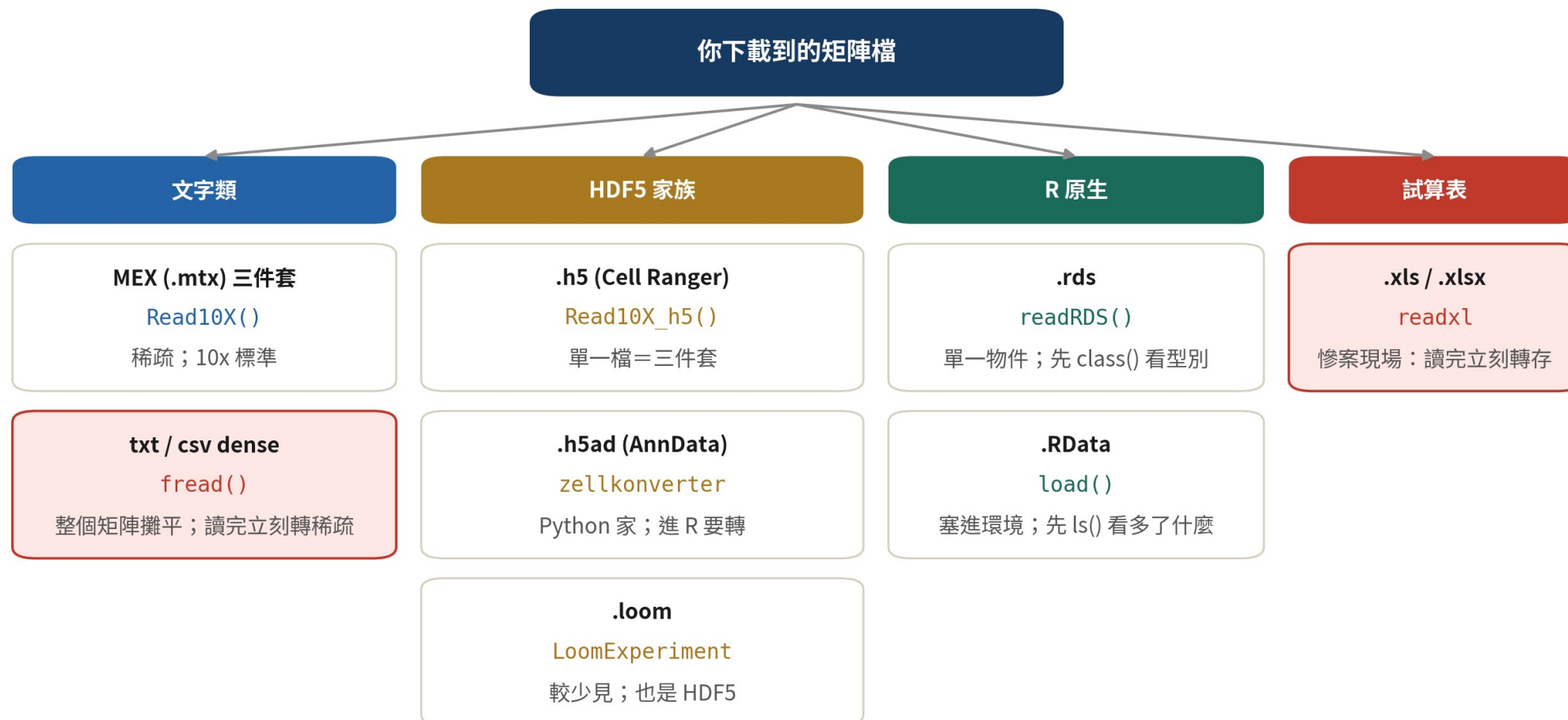
那個檔名誤標
CRC、內容其實是
PDAC—— 下載後照
樣走本集的讀檔五問

03

格式解剖

本集核心：七種穿法，一套認法

格式家族樹



同一種生物內容，七種穿法——認衣服，才知道用哪把鑰匙

內容物都是「基因 × 細胞的數字」——差別只在怎麼包、用什麼函式拆

複習： 10x 三件套（ MEX ）

matrix.mtx. gz



稀疏矩陣本體：只存
非零值的座標與數值
（ B3 的輸出）

features + barcodes



列名與欄名分開存—
—三個檔案缺一不可

Read10X 吃 資料夾



B5 講過：給資料夾
路徑不是檔案路徑；
GEO 版常要先改檔
名

HDF5 之一： Cell Ranger 的 .h5

```
library(Seurat)
# 單一 .h5 檔=三件套打包 (腎癌 GSE159115 就給這種)
m at<-Read10X_h5("GSM_tumor_filtered_feature_bc_matrix.h5")
class(m at)
obj<-CreateSeuratObject(m at,min.cells = 3)
```

```
[1] "dgCMatrix"
attr(,"package")
[1] "Matrix"
```

讀出來直接是稀疏矩陣——.h5 是三件套的單檔版，不是另一個世界

HDF5 家族三兄弟

.h5 (Cell Ranger)



counts 三件套的單檔打包；
Read10X_h5 一行解決

.h5ad (AnnData)



Python 生態的標準物件：矩陣＋metadata＋降維全在裡面；進 R 要轉

.loom



較少見的老格式，也是 HDF5 ；
LoomExperiment 可讀

h5ad 進 R：zellkonverter

```
library(zellkonverter)
# 基礎 20 同款：CELLxGENE 下載的 Tabula Sapiens 胰臟
sce <- readH5AD("TS_pancreas.h5ad") # 回 SingleCellExperiment
assayNames(sce) # 先看有哪些矩陣層
library(Seurat)
obj <- as.Seurat(sce, counts = "X", data = NULL)
```

```
[1] "X"
```

注意：X 是 counts 還是 log 過的值？——讀檔五問的第 4 問

轉檔的坑不在函式，在「X 裡裝的是什麼尺度」——待會第四段專門講

h5ad ↔ Seurat : 路線與現實

主線： zellkonverte



Bioconductor 套件、
走 SCE 中繼站——
課程與題庫的建議路
線

備選： SeuratDisk



h5Seurat 中繼的直
轉路線；版本相容性
要自己盯

趨勢： rds 在 退場



CELLxGENE 的 rds
支援已在棄用中——
R 使用者遲早要會讀
h5ad ，早練早好

dense 文字矩陣： fread ，然後立刻轉稀疏

```
library(data.table); library(Matrix)
x <- fread("GSExxx_counts.txt.gz") # dense : 比 read.csv 快
genes <- x[[1]]; x[,1] := NULL
m <- as.matrix(x); rownames(m) <- genes
sm <- as(m, "CsparseMatrix") # 立刻轉稀疏
rm(x, m); gc() # 放掉 dense , 高峰才會退
```

```
# 高峰出現在轉換瞬間： dense 與稀疏短暫並存
# 所以「讀得進來」的標準是：記憶體 > dense 大小 + 稀疏大小
```

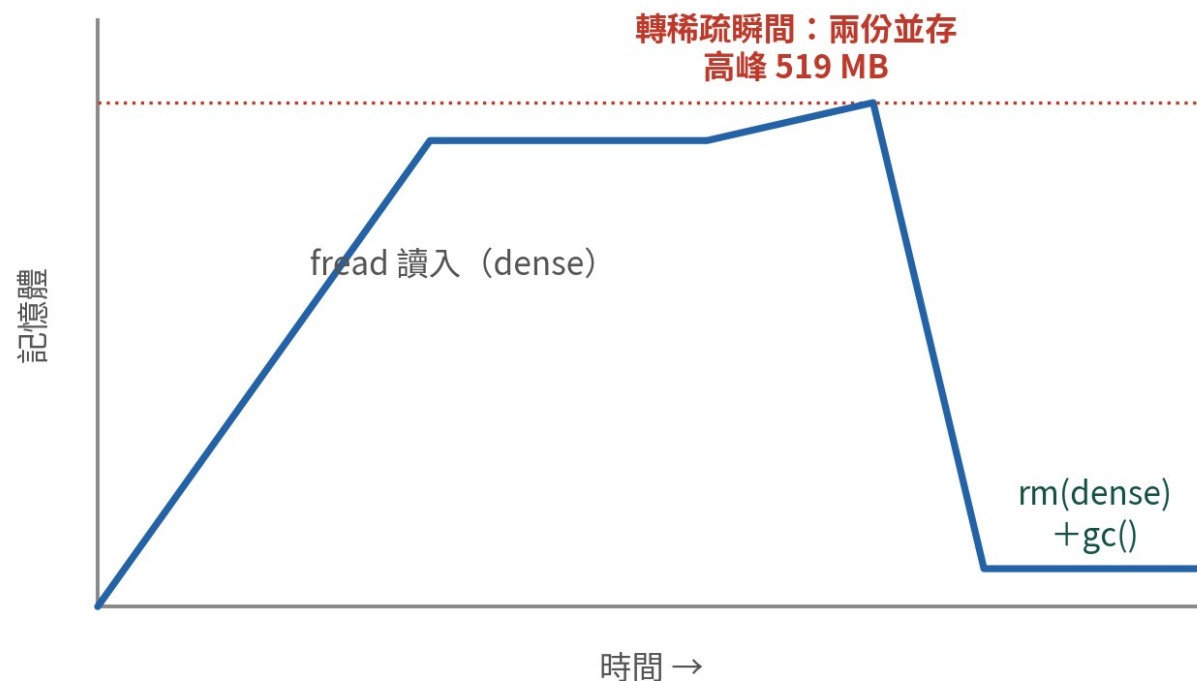
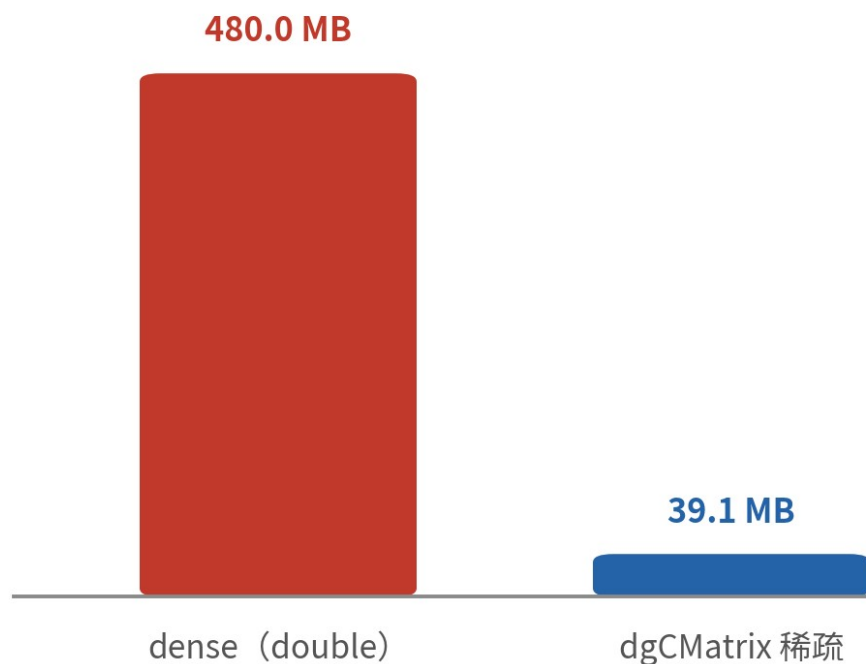
鼻咽癌 GSE150430、皮膚鱗癌 GSE144236 都是這種單一大 dense txt

為什麼要立刻轉稀疏：實測

dense 攤平 vs 稀疏儲存：實測

模擬計數矩陣 20,000 genes $\times$ 3,000 cells：95% 是零

同一份矩陣，小 12 倍



模擬資料

同一份模擬矩陣：dense 480 MB、稀疏 39 MB、轉換高峰 519 MB——先估高峰再動手

R 原生檔： rds 與 RData

rds：先 class()



readRDS 回單一物件——可能是矩陣、Seurat、SCE 或 Monocle3 cds (小鼠 MOCA 就是 cds)

RData：先 ls()



load() 把物件塞進環境——載入前後各 ls() 一次，看多了什麼 (細胞株混合的 RData 給三個物件)

版本注意



老物件常要 UpdateSeuratObject；跨大版本讀不動時，找作者存的矩陣層重建

XLS 慘案

雷是什麼

整個 counts 矩陣存成 Excel 檔——神經母細胞瘤 GSE137804 就是逐樣本 XLS。



為什麼會踩

Excel 會自作主張：SEPT1、MARCH1 這類基因名被改成日期是著名的老災難；讀取又慢又吃記憶體。

踩了會怎樣

紀律：readxl 讀進來、驗過基因名、立刻轉存 rds——之後的流程不再碰 XLS。

readxl 批次轉檔

```
library(readxl)
fs <- list.files("GSE137804", "\\xls$", full.names = TRUE)
for (f in fs) {
  m <- read_excel(f)          # 一個樣本一個 XLS
  stopifnot(!any(grepl("^ [0-9]+ -", m[[1]]))) # 基因名變日期?
  saveRDS(m, sub("\\xls$", ".rds", f))      # 轉存，之後不碰 XLS
}
```

那行 `stopifnot` 是哨兵：長得像「3-Sep」的基因名一出現就停下來

最後一種陷阱：註解列混在矩陣裡

陷阱格式：註解列混在矩陣裡

GSE72056 型——前幾列不是基因，是每個細胞的註解

Cell	cell_1	cell_2	cell_3	...
tumor	53	53	81	...
malignant (1/2/0)	2	1	2	...
non-malignant cell type	0	1	0	...
A1BG	0.0	2.3	0.0	...
A1CF	0.0	0.0	1.1	...
A2M	5.8	0.0	0.0	...

第 2-4 列：細胞註解；第 5 列起才是基因

沒拆就直接算 QC？三列註解會混進每一個統計量

黑色素瘤 GSE72056：前三列是 tumor / malignant / cell type——拆成 metadata 再建物件

註解列 → metadata

tumor / malignant / type
轉成 data.frame，
跟矩陣分家

其餘列 → 表達矩陣

值已是 $\log_2(\text{TPM}/10+1)$
——尺度也要一起判讀
(下一段)

偵察兵: `readLines(n = 10)`

```
# 任何陌生文字檔，先看前幾行再決定怎麼讀  
readLines("GSE72056_melanoma_matrix.txt.gz", n = 4)
```

```
[1] "Cell\tcy53.1\tcy53.2\t..."  
[2] "tumor\t53\t53\t..."  
[3] "malignant(1= no,2= yes,0= unresolved)\t2\t1\t..."  
[4] "non-malignant cell type\t0\t1\t..."
```

四行輸出就把陷阱全曝光了——這是全集性價比最高的一行程式

格式解剖小結

先看檔案，再選函式。

readLines 看文字、class 看 R 物件、assayNames 看 SCE—— 沒有偵察過的檔案不要直接餵進管線。

04

尺度判讀

counts、TPM、log——矩陣不會自我介紹，分布會

同一份資料的三種長相

同一份資料的三種尺度長相

拿到矩陣先畫值的分布——尺度自己會招供

模擬資料

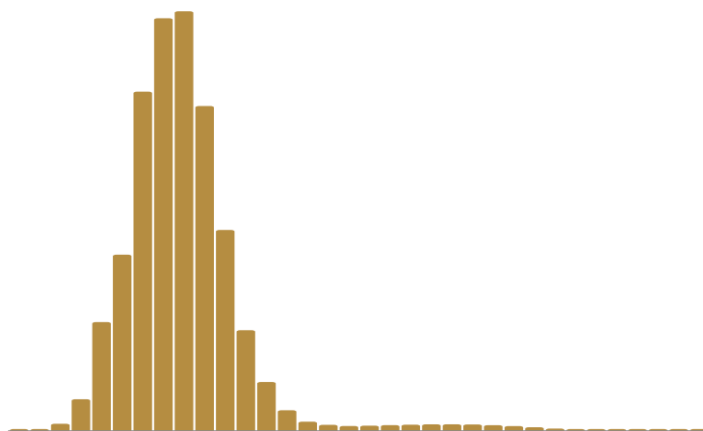
raw counts



值 (log10 刻度)

- 全是整數
- 最大值 1,556
- 零占 95%

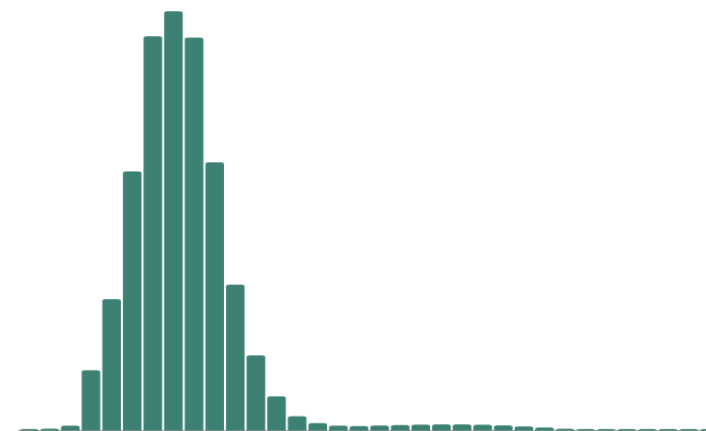
TPM / CPM (未取 log)



值 (log10 刻度)

- 小數，不再是整數
- 最大值 220,031
- 每細胞總和固定 1e6

$\log_2(\text{TPM}/10 + 1)$



值 (線性刻度)

- 被壓進個位數範圍
- 最大值 14.4
- 負值？那是另一種 log

整數與長尾 = counts ; 小數且欄總和固定 = TPM/CPM ; 被壓進個位數 = log 過了

尺度判讀線索表

GEO 頁與論文 Methods 是官方說法；分布是物證——兩邊都要對

尺度	長相線索	下游動作
raw counts	全整數、零很多、最大值成百上千	正常流程：從 NormalizeData 起步
TPM / CPM	小數；每細胞欄總和幾乎相同	library size 已除過——別再除一次；log 自己取
log 尺度	值壓在 0 到十幾；常見 $\log_2(\text{TPM}/10+1)$	放 data 層、絕不再 log ；下游方法的尺度假設逐一過

三行檢查：讓矩陣招供

```
v <- sm @ x          # 稀疏矩陣的非零值
all(v == round(v))    # TRUE → 整數 → counts
max(v)                # 十幾 → 幾乎確定 log 過
head(Matrix::colSums(sm)) # 各欄總和幾乎相同 → TPM / CPM
```

```
[1] TRUE      ← 這份是 counts
[1] 1556
# 若 max 只有 14.4、又不是整數：log 尺度實錘
```

輸出裡的 1556 與 14.4 就是剛才那張模擬圖算出來的值——三行足以定案

尺度判錯的代價

雷是什麼

把 log 過的矩陣塞進 counts 層，或把 TPM 再 NormalizeData 一次。



為什麼會踩

每個下游函式都對輸入尺度有假設——黑色素瘤那份 $\log_2(\text{TPM}/10+1)$ 直接當 counts 用，HVG 與 DE 全部歪掉。

踩了會怎樣

而且不會報錯。錯誤尺度跑出來的圖照樣漂亮，只是結論不可信——跟 B19 的災難同款安靜。

讀檔決策日誌： `decisions.md`

記什麼



凡是「資料逼你做的決定」：格式路線、尺度判定、註解列處理、閾值與理由

固定格式



發現了什麼 → 有哪些選項 → 你選了什麼 → 理由——題庫
「真實數據關卡」同款

它是產出



進階題把它列為繳交物：這份日誌就是論文 Methods 的草稿

本段鐵則

**尺度判錯，下游全錯——
而且不會報錯。**

所以判讀要有物證（分布、三行檢查），且判讀本身要進 decisions.md 。

05

總結：讀檔五問

整集收成一張卡

讀檔五問檢核卡

讀檔五問（每份陌生資料過一遍）

答案全部寫進 decisions.md——它就是你論文 Methods 的草稿



① 這是哪一層？

raw 還是 processed？再分析只要 processed



② 這是什麼格式？

readLines(n=10) 先看長相，再選讀檔函式



③ 稀疏還是 dense？

dense 讀完立刻轉稀疏；先估記憶體高峰



④ 這是什麼尺度？

counts/TPM/log？畫分布佐證，別只信說明



⑤ 有沒有混進非表達列？

註解列、合計列、重複基因名——拆乾淨再建物件

每份陌生資料過一遍五問，答案進 decisions.md——這張卡貼在螢幕旁

四段回顧

段落	一句話帶走
公開資料地圖	看 accession 認出處；三層入口由下往上問
下載實務	先抓小檔、wget -c、md5 對上才算完成；看到管制就繞路
格式解剖	認衣服選鑰匙；dense 立刻轉稀疏；XLS 立刻轉存
尺度判讀	分布會招供；判讀與理由全部進 decisions.md

下一步去哪裡

參考庫



《單細胞與空間資料庫參考庫》：16 個精選入口 + 11 個延伸入口，含決策樹

題庫實戰



基礎 20 題 + 進階 60 題；進階題的「真實數據關卡」就是本集技能的考場

BS2 平台大比較



下一集補充：10x 之外的世界——平台怎麼改變 QC 與分析邏輯

自測 1

論文的 **Data availability** 只寫了「HRA004321」。下一步是？

- A 上 GSA-Human 送出存取申請
- B 寫信請作者直接寄 FASTQ
- C 認出這是管制存取，繞路找處理版（GEO / Zenodo / Dryad / figshare）
- D 放棄這份資料

答 C。HRA 前綴＝GSA-Human 管制區，而管制的幾乎都是 raw。再分析只要 processed：回 GEO 搜、讀 Data availability 找通用儲存庫——進階 13 的 Zenodo 繞路就是範本。

自測 2

fread 讀完一個 2 GB 的 dense txt 矩陣，接下來第一件事是？

- A 直接 CreateSeuratObject
- B 轉成稀疏矩陣，然後 rm 掉 dense 版並 gc()
- C 先跑 NormalizeData
- D 存成 csv 備份

答 B。dense 攤平的矩陣佔記憶體可以差出十倍以上（實測 12 倍），而且轉換瞬間兩份並存——轉稀疏、釋放 dense，之後的一切才有空間跑。

自測 3

矩陣的值全是小數、沒有負值、最大 14.4 。最可能的尺度是？

- A raw counts
- B TPM (未取 log)
- C log 尺度 (如 $\log_2(\text{TPM}/10+1)$)
- D z-score

答 C。counts 是整數、TPM 最大值會到幾萬、z-score 有負值——被壓進十幾的小數只剩 log 一族。確認後放 data 層、絕不再 log ，並把判讀寫進 decisions.md 。

下集預告

BS2：平台大比較—— 10x 之外的世界

同一管血、兩種平台，QC 圖長得完全不一樣——哪個壞掉了？都沒壞。
液滴、微孔、平盤全長、條碼組合：平台是假設的一部分。下一集見。